

Materia: Programación III

Carrera: Tecnicatura Universitaria en Programación (UTN)

Tema: Arquitectura Cliente-Servidor, Transición de C# a JavaScript y Programación Asíncrona

CLASE 1: Arquitectura Cliente-Servidor y fundamentos de JavaScript

1. Arquitectura Cliente-Servidor (Conceptos Fundamentales)

En las materias anteriores (Programación I y II), las aplicaciones ejecutaban toda su lógica, persistencia e interfaz gráfica en una sola máquina (aplicaciones de consola o de escritorio monolíticas). En **Programación III** dividimos el sistema en dos roles principales:

- **El Servidor (Backend):** Responsable de la lógica de negocio, acceso a la base de datos (mediante un ORM) y la seguridad de los datos. Expone servicios (APIs) a los cuales los clientes se conectan.
- **El Cliente (Frontend / Mobile):** Responsable de la interfaz gráfica de usuario (UI), la interacción con el usuario y la presentación visual de la información. En nuestro curso, la aplicación móvil desarrollada con React Native.
- **Protocolo de Comunicación (HTTP / HTTPS):** El canal por el cual se transfieren los datos entre el cliente y el servidor en formato JSON.

CLIENTE (Mobile) (React Native / JavaScript)	Petición HTTP ----->	SERVIDOR (Backend) (.NET / C# Web API)
Interfaz de Usuario (UI)	<----- Respuesta JSON	Lógica + Base de Datos

2. Introducción a JavaScript (JS)

JavaScript es el lenguaje estándar del desarrollo web y móvil multiplataforma (React Native). A diferencia de C#, JavaScript es un lenguaje interpretado de tipado dinámico. Esto significa que las variables no requieren especificar su tipo de dato al declararse.

A. Declaración de Variables

Concepto	C#	JavaScript
Texto	string nombre = "Juan";	let nombre = "Juan";
Números	int edad = 20;	let edad = 20;

Concepto	C#	JavaScript
	float precio = 9.99f;	let precio = 9.99;
Booleano	bool activo = true;	let activo = true;
Inmutabilidad / Constantes	const int MAX = 100;	const MAX = 100;
Variables mutables	int x = 10;	let x = 10; <i>(Evitar el uso de var)</i>

B. Objetos y Estructuras de Datos

En C# definían clases o structs para modelar entidades. En JavaScript utilizaremos **Objetos Literales** para representar estructuradamente los datos recibidos en formato JSON desde el servidor:

C#:

```
public class Usuario {
    public int Id { get; set; }
    public string Nombre { get; set; }
    public string Email { get; set; }
}

Usuario usuarioActual = new Usuario { Id = 1, Nombre = "Osvaldo" };
```

JavaScript:

```
// Creación directa de objeto literal (JS no requiere definir clase previa)
const usuarioActual = {
    id: 1,
    nombre: "Osvaldo",
    email: "osvaldo@ejemplo.com"
};
```

C. Funciones Flecha (Arrow Functions)

En C# usaban métodos tradicionales o expresiones lambda ($\Rightarrow$). En JavaScript, las **funciones flecha** son el estándar moderno para definir funciones:

C#:

```
public int Sumar(int a, int b) {  
    return a + b;  
}
```

JavaScript:

```
const sumar = (a, b) => {  
    return a + b;  
};  
  
// Sintaxis simplificada de una sola línea:  
const sumarCorta = (a, b) => a + b;
```

D. Transformación de Colecciones: Arreglos (Arrays) vs. LINQ

En C# utilizaban **LINQ** (`.Select()`, `.Where()`, `.FirstOrDefault()`). En JavaScript utilizaremos métodos nativos de los arreglos:

```
const numeros = [1, 2, 3, 4, 5];  
  
// 1. Filtrar elementos mayores a 2 (Equivalente a .Where en LINQ)  
const mayoresA2 = numeros.filter(n => n > 2); // [3, 4, 5]  
  
// 2. Transformar elementos (Equivalente a .Select en LINQ)  
const dobles = numeros.map(n => n * 2); // [2, 4, 6, 8, 10]  
  
// 3. Buscar el primer elemento que cumpla la condición (Equivalente  
a .FirstOrDefault en LINQ)  
const primerPar = numeros.find(n => n % 2 === 0); // 2
```

● CLASE 2: Asincronismo, Promesas y Peticiones HTTP

1. Entendiendo el Asincronismo (Event Loop vs. Hilos)

- **En C#:** Se utiliza ejecución multihilo (Multithreading). Si se realiza una operación bloqueante en el hilo principal de la interfaz (UI Thread), la pantalla se congela.
- **En JavaScript:** El entorno ejecuta sobre un **único hilo principal** (Single-Threaded). Para evitar congelar la interfaz de la aplicación mientras se espera una respuesta del servidor, las operaciones de red y disco se ejecutan de forma **Asíncrona**.

2. De Task<T> en C# a Promise en JavaScript

En C#, para manejar operaciones asíncronas utilizaban Task o Task<T>. En JavaScript utilizamos el objeto **Promise (Promesa)**.

Una Promesa representa un valor que estará disponible inmediatamente, en el futuro o nunca. Atraviesa 3 estados:

1. **Pending (Pendiente):** Estado inicial, esperando la respuesta del servidor o proceso.
2. **Fulfilled / Resolved (Resuelta):** La operación finalizó con éxito y retornó el resultado.
3. **Rejected (Rechazada):** La operación falló (error de red, servidor fuera de línea, etc.).

3. Manejo Moderno de Asincronismo: async / await

La sintaxis de async/await en JavaScript es prácticamente idéntica a la de C#.

C# (.NET HttpClient):

```
public async Task ObtenerDatosAsync() {
    try {
        var client = new HttpClient();
        string respuesta = await
client.GetStringAsync("https://api.ejemplo.com/usuarios");
        Console.WriteLine(respuesta);
    }
    catch (Exception ex) {
        Console.WriteLine($"Error: {ex.Message}");
    }
}
```

JavaScript (fetch API):

```
const obtenerUsuarios = async () => {
    try {
        // Petición HTTP GET asíncrona
        const respuesta = await fetch("https://api.ejemplo.com/usuarios");
    }
}
```

```

    if (!respuesta.ok) {
      throw new Error(`Error en el servidor: ${respuesta.status}`);
    }

    // Deserialización del JSON
    const usuarios = await respuesta.json();
    console.log("Usuarios recibidos:", usuarios);

  } catch (error) {
    console.error("Error al realizar la petición:", error.message);
  }
};

// Llamada a la función asíncrona
obtenerUsuarios();

```

4. Guía Práctica de Verbos HTTP

El intercambio de información entre el Cliente y el Servidor sigue la arquitectura de servicios REST:

Verbo HTTP	Operación CRUD	Descripción
GET	Read (Leer)	Solicita información al servidor. No envía cuerpo de mensaje (body).
POST	Create (Crear)	Envía una nueva entidad en el cuerpo (body) de la petición para crearla.
PUT	Update (Actualizar)	Reemplaza o actualiza un registro completo existente en el servidor.
DELETE	Delete (Eliminar)	Remueve un registro específico del servidor identificándolo por ID.

5. Ejercicio Práctico Sugerido (Ejecutable con Node.js)

Consigna:

1. Crear un archivo llamado ejercicio.js en VS Code.
2. Crear una función asíncrona obtenerTarea(id).
3. Consumir el endpoint público de prueba <https://jsonplaceholder.typicode.com/todos/1> utilizando fetch y async/await.
4. Manejar los posibles errores mediante bloques try/catch e imprimir el título de la tarea en la consola.
5. Ejecutar el archivo desde la terminal de VS Code con el comando: `node ejercicio.js`.